

Lab 4, Act 2 – Answer Key

How much monitoring is enough?

NRES 746 – Instructor Answer Key (not for student distribution)

This Act is a group activity throughout Part 2, so expect one shared set of answers and one shared R script per group rather than individual variation – grade the group’s packet(s) and script together. Part 1 stays individual-first, so some variation there is expected and fine. Questions with a single correct numeric answer are answered directly; open-ended items (1a’s real-world question, 1c’s reasoning, 1d’s predictions, the Reflection) get discussion points rather than a fixed rubric. Every number below comes from the four functions exactly as specified in the handout, so a group’s own numbers should land close to these (Monte Carlo noise aside).

Part 1: Plan the pieces

1a. Not detected on a single person-day: $1 - 0.02 = 0.98$. Not detected across k independent person-days: 0.98^k . So:

$$q(k) = 1 - 0.98^k$$

```
detect_prob <- function(k, pobs = 0.02) 1 - (1 - pobs)^k  
detect_prob(3)
```

```
## [1] 0.058808
```

$q(3) \approx 0.0588$, matching the check value.

Real-world differences between “5 observers, 1 day” and “1 observer, 5 days” (open-ended). Good answers name at least one of: a single observer working 5 days might get better at reading sign/habitat as the days go on (learning), while 5 observers on 1 day bring more independent eyes but no time to adapt; weather or animal activity can vary day to day, so spreading effort over multiple days hedges against one bad day, while concentrating it in one day risks a single unlucky day wasting the whole bout; animals might become wary of repeated visits (habituation or avoidance) if the same day is revisited, which a single-day multi-observer bout avoids entirely. The formula treats all of this as irrelevant because it only assumes each person-day is an independent, identical 2% trial – any of the above would violate that independence or identical- probability assumption.

1b.

$$N_t = N_0 \lambda^t = 1000 \times \left(0.75^{1/25}\right)^t$$

```
true_abundance <- function(t, N0 = 1000, lambda = 0.75^(1/25)) N0 * lambda^t  
true_abundance(c(0, 25))
```

```
## [1] 1000 750
```

$N_{25} = 750$, matching the check value (a clean number since $\lambda^{25} = 0.75$ exactly).

1c. Binomial, with size N_t and probability $q(k)$: $I_t \sim \text{Binomial}(N_t, q(k))$. Reasoning: at year t there are N_t animals present (a fixed number of “trials,” conditional on N_t), each independently detected or not with the same probability $q(k)$, and the survey count is the number of “successes” out of those N_t trials – exactly

the setup the binomial was built for (same reasoning as the Week 5 WOD's nest-fate example, just with N_t trials instead of one).

Common mistake to watch for: proposing a Poisson here. Poisson is a reasonable answer if a group treats N_t as unknown/large and $q(k)$ as small (the usual binomial-to-Poisson limit), and should get credit if they explain that reasoning – but the more direct, exact answer, given that N_t is already known in this simulation, is binomial.

1d. *Two predictions (no fixed answer – these are hunches to test in 2f).*

- **k vs. s :** either direction is a reasonable guess, and a good answer just needs a reason (“more effort per bout should give a cleaner count,” or “more surveys give the regression more points to work with”). The actual finding from 2f: in most of the design space the group will explore, k and s matter by *comparable* amounts, not one dominating the other. For instance, starting from $k = 2, s = 1$ (power ≈ 0.66), doubling k to 4 raises power to about 0.92 (+0.26), while doubling s to 2 lowers it to about 0.38 (−0.28) – almost symmetric. (For instructor reference, not something groups need to derive: with $q(k)$ small, the delta-method variance of a log count scales roughly as $1/k$, while S_{tt} – and hence $\text{Var}(\hat{b})$ – scales roughly as $1/s$, so the two enter the precision of $\hat{b}$ in a similar way.) Give full credit for a reasoned guess in either direction; the point of 1d is committing to a prediction before simulating, not getting it right.
- **False-decline rate at $\lambda = 1$:** should **not** be exactly 5%. The rule requires the slope to be negative *and* significant. Under a truly flat population, a two-sided test is significant about 5% of the time, split roughly evenly between “significantly negative” and “significantly positive” slopes – so requiring *both* negative and significant should roughly halve the false-decline rate to around 2.5%. A simulation check confirms this:

```
simulate_counts <- function(t_survey, k, NO = 1000, lambda = 0.75^(1/25), pobs = 0.02) {
  N_t <- true_abundance(t_survey, NO, lambda)
  rbinom(length(t_survey), size = round(N_t), prob = detect_prob(k, pobs))
}
detect_decline <- function(t_survey, counts, alpha = 0.05) {
  m <- lm(log(counts) ~ t_survey)
  s <- summary(m)$coefficients[2, ]
  unname(s[1] < 0 && s[4] <= alpha)
}
set.seed(14)
t1 <- seq(1, 25, by = 1)
mean(replicate(3000, detect_decline(t1, simulate_counts(t1, k = 3, lambda = 1))))

## [1] 0.023
```

About 2%, in line with the halved-5% reasoning above. **Common mistake to watch for:** a group that predicts exactly 5% is forgetting the rule has two conditions, not one – worth connecting back to why 2d's decision rule needed both.

Group check-in

No fixed answer – the point is to reconcile 1a-1c and agree on both 1d predictions (even if the predictions turn out wrong later) before opening laptops.

Part 2: Build, test, and search

This section is graded as one shared group product: one filled-in result per item (from any group member's packet) plus the group's single emailed R script. All four functions below are given in full, both as a reference for checking a group's script and as a reference implementation for reproducing every number in this key.

```

detect_prob <- function(k, pobs = 0.02) {
  1 - (1 - pobs)^k
}

true_abundance <- function(t, NO = 1000, lambda = 0.75^(1/25)) {
  NO * lambda^t
}

simulate_counts <- function(t_survey, k, NO = 1000, lambda = 0.75^(1/25), pobs = 0.02) {
  N_t <- true_abundance(t_survey, NO, lambda)
  q <- detect_prob(k, pobs)
  rbinom(length(t_survey), size = round(N_t), prob = q)
}

detect_decline <- function(t_survey, counts, alpha = 0.05) {
  m <- lm(log(counts) ~ t_survey)
  s <- summary(m)$coefficients[2, ]
  unname(s[1] < 0 && s[4] <= alpha)
}

```

2a. `detect_prob(3) = 0.0588` (from 1a).

2b. `true_abundance(0) = 1000` (from 1b). As a **vector call**, `true_abundance(c(0, 25))` should return 1000 and 750 *together*, confirming the function handles a whole vector of years, not just one at a time – essential, since `simulate_counts()` hands it the full `t_survey` vector in 2c.

Common mistake to watch for: a `true_abundance()` written with an `if (length(t) == 1)` branch, or one that otherwise only tested against single years, can look correct on the two scalar checks above and still break (or silently return a recycled single value) once it's handed a 25-element `t_survey` vector inside `simulate_counts()`. The vector check is what catches this before it causes a confusing downstream bug.

2c. A representative run (`t_survey = seq(1, 25, by = 1)`, `k = 3`):

```

set.seed(11)
simulate_counts(seq(1, 25, by = 1), k = 3)

```

```
## [1] 65 49 47 55 54 59 44 51 50 57 47 60 49 43 57 57 54 53 51 45 46 52 46 46 42
```

Counts should trend downward (from around 60-70 early on to around 40-50 by year 25) and stay far below the true abundance ($N_0 = 1000$ falling to $N_{25} = 750$) – detection probability at $k = 3$ is under 6%, so only a small fraction of the population is ever counted. The output should also be a **vector of 25 counts**: `length(simulate_counts(seq(1, 25, by = 1), k = 3)) = 25`, matching `length(t_survey)`. **Common mistake to watch for:** a group whose counts *don't* trend downward at all, whose counts are implausibly close to the true abundance, or whose output is a single recycled number rather than 25 distinct values, likely has `detect_prob()` or `true_abundance()` wired in backwards, has a `true_abundance()` that isn't actually vectorized (see 2b), or forgot to round `N_t` before passing it to `rbinom()`'s `size` argument (which requires an integer and silently truncates otherwise).

2d.

```

set.seed(12)
t1 <- seq(1, 25, by = 1)
mean(replicate(300, detect_decline(t1, simulate_counts(t1, k = 3))))

```

```
## [1] 0.8166667
```

```
mean(replicate(300, detect_decline(t1, simulate_counts(t1, k = 3, lambda = 1))))
```

```
## [1] 0.02
```

Under the real decline, `detect_decline()` should return `TRUE` a clear majority of the time (roughly 80% at $k = 3$, $s = 1$ – see 2e/2f); under $\lambda = 1$ it should return `TRUE` only rarely (around 2-3%, matching 1d). **Common mistake to watch for:** a `detect_decline()` that checks only the p -value (forgetting the sign of the slope) will falsely detect a decline roughly twice as often under the null as it should (closer to 5% than 2.5%), since it also counts significant *increases* as “detected declines.”

2e.

```
estimate_power <- function(k, s, R = 1000, lambda = 0.75^(1/25), alpha = 0.05,
                           N0 = 1000, pobs = 0.02) {
  t_survey <- seq(1, 25, by = s)
  mean(replicate(R, {
    counts <- simulate_counts(t_survey, k, N0, lambda, pobs)
    detect_decline(t_survey, counts, alpha)
  })))
}
set.seed(14)
fd_rate <- estimate_power(3, 1, R = 3000, lambda = 1)
fd_rate
```

```
## [1] 0.023
```

```
sqrt(fd_rate * (1 - fd_rate) / 3000) # Monte Carlo SE
```

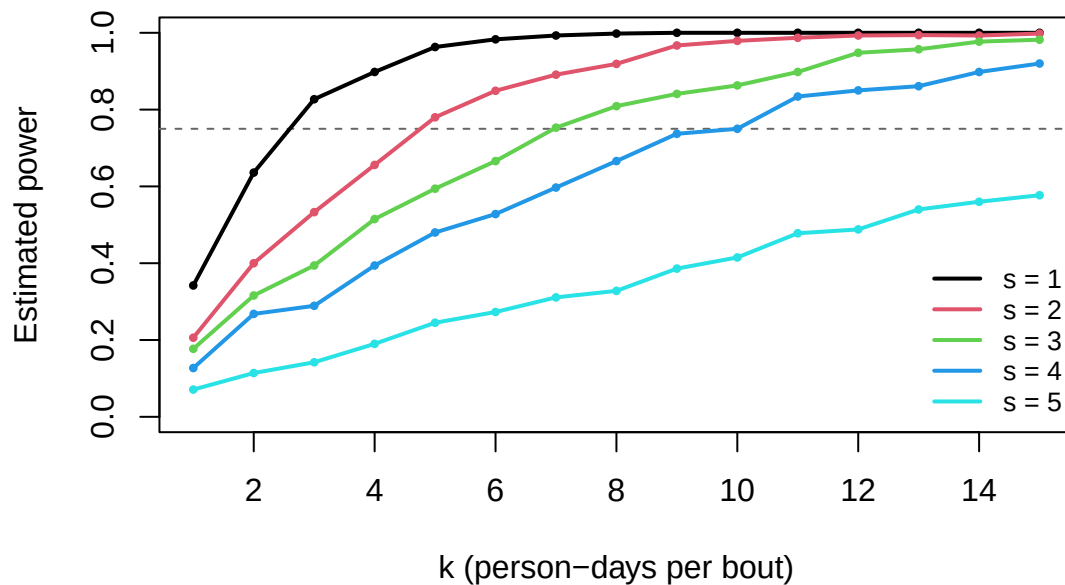
```
## [1] 0.002736847
```

False-decline rate ≈ 0.02 - 0.03 , consistent with 1d’s halved-5% prediction.

2f.

```
set.seed(746)
grid <- expand.grid(k = 1:15, s = 1:5)
grid$power <- mapply(estimate_power, grid$k, grid$s, MoreArgs = list(R = 1000))

plot(NA, xlim = c(1, 15), ylim = c(0, 1), xlab = "k (person-days per bout)",
     ylab = "Estimated power")
abline(h = 0.75, lty = 2, col = "grey40")
for (ss in 1:5) {
  d <- grid[grid$s == ss, ]
  lines(d$k, d$power, col = ss, lwd = 2)
  points(d$k, d$power, col = ss, pch = 16, cex = 0.6)
}
legend("bottomright", legend = paste("s =", 1:5), col = 1:5, lwd = 2, bty = "n", cex = 0.8)
```



Pattern to expect: power rises with k for every survey interval s , and every curve shifts down and to the right as s increases – annual surveys ($s = 1$) cross the 0.75 line around $k = 3$, while every-5-year surveys ($s = 5$) haven't reached it even at $k = 15$. As discussed in 1d, k and s trade off at roughly comparable rates through most of this grid (neither dominates outright), though $s = 1$ is clearly the most power-efficient interval on its own.

2g.

```
cost_of <- function(k, s, nyears = 25) length(seq(1, nyears, by = s)) * (2000 + 200 * k)
grid$cost <- mapply(cost_of, grid$k, grid$s)
ok <- grid[grid$power >= 0.75, ]
ok <- ok[order(ok$cost), ]
head(ok, 6)
```

```
##      k s power  cost
## 55 10 4 0.750 28000
## 56 11 4 0.834 29400
## 37  7 3 0.753 30600
## 57 12 4 0.850 30800
## 58 13 4 0.861 32200
## 38  8 3 0.809 32400
```

```
top3 <- head(ok, 3)
set.seed(999)
top3$power_recheck <- mapply(estimate_power, top3$k, top3$s, MoreArgs = list(R = 5000))
top3$mc_se <- sqrt(top3$power_recheck * (1 - top3$power_recheck) / 5000)
top3
```

```
##      k s power  cost power_recheck      mc_se
## 55 10 4 0.750 28000      0.7568 0.006067186
## 56 11 4 0.834 29400      0.8052 0.005600946
```

```
## 37 7 3 0.753 30600 0.7380 0.006218617
```

k	s	Cost	Power (re-checked)	MC SE
10	4	\$28,000	≈ 0.76	$\approx 0.004-0.006$
11	4	\$29,400	$\approx 0.80-0.81$	$\approx 0.004-0.006$
7	3	\$30,600	$\approx 0.74-0.75$	$\approx 0.004-0.006$

This is the payoff of the “re-check with more replicates” instruction. The cheapest design from the 1,000-replicate grid ($k = 10, s = 4$, \$28,000) sits *right on* the 0.75 threshold; a high-precision check ($R = 10,000$) puts it at about 0.764 – it does clear the bar, but with almost no margin. The third-cheapest candidate ($k = 7, s = 3$, \$30,600) looked like it cleared 0.75 in the 1,000-replicate grid (0.753) but drops to about **0.743 – below threshold** – under a high-precision re-check. That’s not Monte Carlo bad luck resolving itself; it’s a genuine case where the coarse grid estimate was on the wrong side of the line.

Recommended design: $k = 11, s = 4$ (**\$29,400**). It costs \$1,400 more than the cheapest candidate but clears 0.75 with real margin (power $\approx 0.80-0.81$, several Monte Carlo standard errors above the threshold), rather than sitting on the boundary where the “true” power could plausibly be under 0.75. **Grading note:** a group that recommends $k = 10, s = 4$ isn’t necessarily wrong – it’s a defensible “minimum cost that clears the bar” choice – but full credit requires them to have actually re-checked it with more replicates and to say something about why they’re comfortable (or not) with how close to the threshold it sits. A group that skips the re-check step entirely and just reports the 1,000-replicate numbers should be asked to go back and do it – that’s the whole point of 2g’s instruction.

If you finish early (environmental noise).

```
true_abundance_env <- function(t, NO = 1000, lambda = 0.75^(1/25), sd_env = 0.05) {
  maxt <- max(t)
  eps <- rnorm(maxt, 0, sd_env)
  logN <- log(NO) + cumsum(rep(log(lambda), maxt) + eps)
  c(NO, exp(logN))[t + 1]
}

simulate_counts_env <- function(t_survey, k, NO = 1000, lambda = 0.75^(1/25),
                                pobs = 0.02, sd_env = 0.05) {
  N_t <- true_abundance_env(t_survey, NO, lambda, sd_env)
  rbinom(length(t_survey), size = round(N_t), prob = detect_prob(k, pobs))
}

estimate_power_env <- function(k, s, R = 2000, lambda = 0.75^(1/25), alpha = 0.05, sd_env = 0.05) {
  t_survey <- seq(1, 25, by = s)
  mean(replicate(R, {
    counts <- simulate_counts_env(t_survey, k, lambda = lambda, sd_env = sd_env)
    detect_decline(t_survey, counts, alpha)
  })))
}

set.seed(748)
estimate_power_env(11, 4) # power with environmental noise, recommended design

## [1] 0.499

set.seed(750)
estimate_power_env(11, 4, lambda = 1) # false-decline rate with environmental noise

## [1] 0.1545
```

For the recommended design, power with environmental noise drops from about 0.80 to roughly **0.50**, and the false-decline rate rises from about 2-3% to roughly **15%**. The assumption from Act 1 that’s now violated is **independence of the year-to-year noise terms** – Act 1’s model had ε_t drawn independently every

year, but persistent shocks mean this year's deviation carries into next year's abundance, so consecutive counts are correlated rather than independent. This is exactly why the false-decline rate roughly quintuples: a single bad (or good) run of years now looks like a real trend to a method that assumes independent noise. The takeaway worth drawing out: real monitoring data plausibly needs a model that accounts for this kind of temporal autocorrelation (a preview of time-series/state-space content later in the course), not just more replicate surveys of the same naive model.

Reflection

No fixed answers for R1-R4 – these are individual (per the house note at the top of the Act) and should be graded for engagement, not a specific “correct” response. A few things worth recognizing as strong answers:

- **R1:** names the actual recommended design and cost, and identifies a real assumption (e.g., independent detection across person-days, no environmental persistence, known and constant N_0) rather than a generic “the model could be wrong.”
- **R2:** written for a non-technical reader – no jargon like “power,” “ p -value,” or “Monte Carlo” without a plain-language gloss.
- **R3:** no single correct answer, but a strong response recognizes *why* grid search is reasonable here specifically, rather than either defending or dismissing it in the abstract: the design space is small (75 combinations of k and s) and both variables are naturally discrete – you can't run “3.7 person-days” or survey “every 2.3 years” – so exhaustive search is cheap and directly respects the real decision space. Grid search also produces the whole power surface (2f's plot), which is useful for seeing trade-offs and explaining the recommendation to a non-technical stakeholder, not just a single “optimal” point.

For “better ways,” reasonable answers include: an adaptive/staged search that runs every grid point cheaply first and only re-checks the near-threshold candidates with many more replicates (this is literally what 2g already does, just not generalized into a search strategy); exploiting that power is roughly monotonic in both k and s to search more cleverly (e.g., something like a binary search along each row instead of evaluating every cell); or a numerical optimizer such as `optim()` if k and s were treated as continuous and then rounded, though for a search space this small that adds complexity without an obvious payoff. Worth surfacing explicitly: grid search stops being practical once the design space grows past a couple of dimensions (adding, say, survey season or number of sites would make the grid explode combinatorially), which is when a smarter method actually earns its keep. **Common misconception to watch for:** claiming grid search is “obviously worse” than some other named method without weighing that claim against the size and discreteness of *this* particular design space – there is no universally better search method, only one better suited to a given problem's dimensionality and structure.

- **R4:** names something concrete the group had to fix or push back on (a wrong argument order, a forgotten `round()`, an AI-suggested s range that didn't match the handout's), not just “it worked great.”